End-to-End Data Science Journey: A Case Study Using SpaceX Launch Data



Name: Alan Ganesh
Date: November 8, 2026

IBM Developer
SKILLS NETWORK

OUTLINE

End-to-End Data Science Journey: A Case Study Using SpaceX Data

1 **Executive Summary**

2 Introduction

3 Methodology

4 Results

5 **Conclusion**

EXECUTIVE SUMMARY

End-to-End Data Science Journey: A Case Study Using SpaceX Data

This case study demonstrates an end-to-end data science workflow using SpaceX launch data. The project showcases how data science tools can be integrated into a reproducible process for extracting insights and building predictive models.

METHODOLOGY & WORKFLOW

- Data Collection: Acquire SpaceX launch data from APIs and web scrape Falcon9 launch record from Wikipedia
- Data Preparation: Clean, integrate, and transform data for exploratory data analysis (EDA).
- EDA: Explore distributions, trends, relationships, and launch outcomes using SQL queries.
- Features engineering: Select and construct variables relevant to prediction.
- Interactive Visualization: Mapping using Folium and interactive dashboard with Plotly Dash.
- Predictive Analysis: Machine learning pipeline-based model building with Logistic Regression, SVM, Decision Tree, K-Nearest Neighbors (KNN), in conjunction and tuning hyperparameters. Evaluate models using test data accuracy.

KEY FINDINGS & TAKEAWAYS

- Data and EDA insights: Identification of key features influencing Flacon 9 launch success rates.
- Visualization insights: Facilitated quick Identification of success and failure rates based on geographical and payload.
- Predictive performance with test data: Decision Tree model provided excellent (94.44%) accuracy compared to very good (83.33%) accuracy of Logistic Regression, SVM and KNN models.
- Improvement of success rate over time.
- Payloads and launch sites played key factors for landing success.
- The supervised machine learning Decision Tree is the most effective predictor of the landing outcomes.

INTRODUCTION

End-to-End Data Science Journey: A Case Study Using SpaceX Data

CAPSTONE PROJECT BACKGROUND & MOTIVATION

SpaceX has helped transform the commercial space industry through reusable launch technology. The Falcon 9 first stage is designed to return to Earth and, when successfully recovered, can be refurbished and flown again. Reusability can substantially reduce launch costs.

This creates a useful data science problem: can publicly available launch data be used to predict whether a Falcon 9 first stage will land successfully—and therefore be available for potential reuse? This project applies an end-to-end data science workflow to investigate that question.

PROJECT OBJECTIVES

- Collect and prepare publicly available SpaceX launch data.
- Explore patterns and relationships through EDA and visualization.
- Develop and compare machine learning models for binary classification.
- Evaluate model performance and identify useful predictors of landing success.

RESEARCH QUESTIONS

- How do payload mass, launch site, flight number, and orbit relate to landing success?
- Has the first-stage landing success rate increased over time?
- Which binary-classification algorithm provides the most effective predictions?



Methodology

Data Collection

- SpaceX launch data from public sources

Data Wrangling & Preparation

- Clean data
- Handle missing values
- Engineer features

Exploratory Data Analysis

- Visualize data
- Identify key patterns

Machine Learning

- Apply classification algorithms
- Tune models

Evaluation

- Assess performance (accuracy, precision, etc.)
- Interpret results

An end-to-end data science workflow to predict Falcon 9 first-stage landing success

METHODOLOGY

DATA ACQUISITION & PREPARATION

1 Data Collection

- Retrieve launch records from multiple SpaceX API endpoints.
- Consolidate API responses into a unified dataset.
- Scrape tabular launch information from Wikipedia.
- Extract the records and fields relevant to first-stage landing outcomes.

2 Data Wrangling & Preparation

- Flatten nested fields and standardize data formats.
- Identify and handle missing values; resolve inconsistencies.
- Select relevant variables and engineer additional features.
- Prepare a clean, analysis-ready dataset for EDA and modeling.

OUTPUT | A consolidated, cleaned dataset ready for analysis

METHODOLOGY

EXPLORATION, VISUAL ANALYTICS & PREDICTION

3 Exploratory Data Analysis

- Visualize launch success, payloads, and launch sites using Matplotlib and Seaborn.
- Use SQL queries to answer focused questions.
- Examine aggregate trends and relationships among variables.

4 Interactive Visual Analytics

- Use Folium to map launch sites and mark successful and failed launches.
- Calculate distances to nearby locations.
- Build a Plotly Dash app with dropdowns and sliders for interactive exploration.

5 Predictive Analysis

- Train binary classifiers: Logistic Regression, SVM, KNN, and Decision Trees.
- Tune hyperparameters using GridSearchCV.
- Evaluate and compare models using accuracy; identify the best-performing model.

GitHub URL : https://github.com/algan12/DS-capstone

DATA COLLECTION

Combining SpaceX REST API records with Wikipedia launch tables

1

SPACEX API REQUEST

- Initiate API request
- Fetch Falcon 9 launch data
- Store API responses locally

Structured launch records

2

WIKIPEDIA WEB SCRAPING

- Extract the HTML launch table
- Parse table content with BeautifulSoup
- Convert extracted records to a DataFrame

Supplementary launch details

3

DATA INTEGRATION

- Standardize fields and formats
- Align records using flight number and date
- Merge API and Wikipedia datasets

Final integrated dataset

KEY DATA | Launch date • Flight number • Booster version • Payload mass • Orbit • Launch site • Landing outcome • Reuse information

DATA COLLECTION – SPACEX API

Retrieving, transforming, and preparing Falcon 9 launch records with Python

1

INITIATE API REQUEST

- Use Python requests.get()
- GET /v4/launches
- Retrieve launch records

SpaceX REST API • HTTP GET

2

PARSE API RESPONSE

- Decode JSON with response.json()
- Extract fields using custom functions
- Normalize with pandas.json_normalize()

Date • Site • Payload • Rocket • Outcome

3

PREPARE & STORE DATA

- Filter for Falcon 9 launches
- Fill missing payload mass with mean
- Build DataFrame; export to CSV

pandas.DataFrame → data.csv

FLOW

API GET → JSON PARSE → EXTRACT FIELDS → CLEAN + FILTER → CSV EXPORT

GitHub URL file: jupyter-labs-spacex-data-collection-api.ipynb

DATA COLLECTION – SCRAPING

Retrieving, parsing, and structuring Falcon 9 launch tables from Wikipedia

1

INITIATE WEB SCRAPING

- Use Python requests.get()
- Fetch HTML from Wikipedia
- Target the Falcon 9 / Falcon Heavy launches page

Wikipedia page • HTTP GET

2

PARSE HTML CONTENT

- Parse page content with BeautifulSoup
- Locate the Falcon 9 launch table
- Extract launch records from the HTML table

BeautifulSoup • HTML table extraction

3

CONVERT TO DATAFRAME

- Convert the HTML table to pandas.DataFrame
- Drop nulls and standardize columns
- Clean and format data for consistency

pandas.DataFrame → cleaned launch data

FLOW

REQUEST PAGE → PARSE HTML → EXTRACT TABLE → TO DATAFRAME → CLEAN + FORMAT

GitHub URL file: jupyter-labs-webscraping.ipynb

DATA WRANGING

Explore Falcon 9 launches, interpret booster landings, and generate binary training labels

1

EXPLORE LAUNCH DATA

- Count launches at each site
- Count launches by orbit type
- Analyze landing outcomes within each orbit

Site • Orbit • Outcome distributions

2

INTERPRET OUTCOMES

- Ocean: landing in an ocean region
- RTLS: return to ground landing pad
- ASDS: landing on a drone ship
- True = success; False = failure

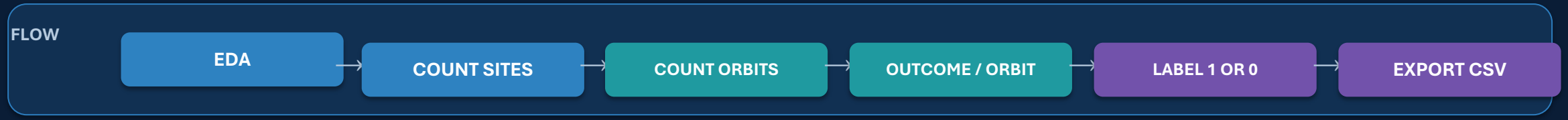
True/False × Ocean / RTLS / ASDS

3

CREATE TRAINING LABELS

- Read the Outcome column
- Assign 1 to successful landings
- Assign 0 to unsuccessful landings
- Export labeled data to CSV

Landing outcome → Class {0, 1}



GitHub URL file: labs-jupyter-spacex-Data wrangling.ipynb

EDA WITH DATA VISUALIZATION

Exploring launch performance, variable relationships, and trends

1

SCATTER PLOTS

- Flight number vs. payload mass
- Flight number vs. launch site
- Payload mass vs. launch site
- Flight number / payload mass vs. orbit type

Relationships • patterns • outliers

2

BAR & DISTRIBUTION CHARTS

- Orbit type vs. success rate
- Compare launch sites and outcomes
- Examine flight number and payload distributions
- Identify category frequencies

Category comparisons • frequencies

3

LINE CHARTS & TRENDS

- Visualize yearly success rate
- Track changes in landing performance
- Find temporal patterns and trends
- Inform later feature selection

Success rate • yearly trend

FLOW

EXPLORE DATA → PLOT RELATIONS → COMPARE → TRACK TRENDS → FIND FEATURES

GitHub URL file: edadataviz.ipynb

EDA WITH DATA VISUALIZATION

From exploratory patterns to machine-learning-ready input features

1

REVIEW EDA FINDINGS

- Inspect flight number, payload mass, and success
- Compare launch sites and orbit types
- Examine landing outcomes by orbit
- Identify trends and candidate predictors

EDA • distributions • relationships

2

BUILD FEATURE DATAFRAME

- Create the features DataFrame
- Select categorical columns:

  Orbits, LaunchSite, LandingPad, Serial
- Keep rows aligned to training labels

features • 4 categorical columns

3

ONE-HOT ENCODING

- Apply OneHotEncoder to categorical fields
- Create indicator columns for category values
- Assign encoded data to features_one_hot
- Use encoded features for ML models

features_one_hot • model inputs

FLOW

EDA REVIEW → SELECT FIELDS → BUILD FEATURES → ONE-HOT ENCODE → MODEL INPUTS

GitHub URL file: edadataviz.ipynb

EDA WITH SQL

SQL exploration of Falcon 9 launch sites, payloads, boosters, and outcomes

1

LAUNCH SITES & PAYLOADS

- List unique launch-site names
- Show 5 records with site prefix “CCA”
- Sum NASA (CRS) payload mass
- Average payload for F9 v1.1

DISTINCT • LIKE • SUM • AVG

2

LANDING SUCCESS & BOOSTERS

- Find first successful ground-pad landing
- List successful drone-ship boosters with payload >4,000 and <6,000
- Count successful and failed outcomes
- Find booster versions with max payload

WHERE • COUNT • MAX • GROUP BY

3

DATE FILTERS & RANKING

- Find failed drone-ship landings in 2015, with booster and site
- Rank landing outcomes by frequency, descending
- Date range: 2010-06-04 to 2017-03-20

DATE FILTER • ORDER BY DESC

FLOW

EXPLORE SITES → AGGREGATE → FILTER → GROUP RESULTS → RANK OUTCOMES

GitHub URL file: jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL

Organizing launch-data analysis by SQL query technique

1

AGGREGATION & FILTERING

- COUNT launches and success / failure outcomes
- SUM payloads; AVG payload mass
- Compare success rates by site or rocket
- Filter on site, landing status, date, and payload

COUNT • SUM • AVG • WHERE

2

JOINS & SUBQUERIES

- Join launch records with related rocket details, when stored separately
- Combine datasets for wider analysis
- Use nested queries for derived statistics, e.g., average mass by site

JOIN • SELECT within SELECT

3

SORTING & RANKING

- Order records by launch date or metric
- Group landings by outcome category
- Rank outcome counts from highest to lowest
- Surface trends and unusual records

GROUP BY • ORDER BY • DESC

FLOW

SELECT DATA → FILTER ROWS → JOIN / GROUP → COMPUTE METRICS → SORT RESULTS

BUILD AN INTERACTIVE MAP WITH FOLIUM

Mapping SpaceX launch sites, landing outcomes, and nearby geographic features

1

MAP LAUNCH SITES

- Mark NASA Johnson Space Center as a reference
- Add site markers, circles, popup and text labels
- Compare distances to equator and coastlines

Coordinates • Markers • Labels

2

COLOR LANDING OUTCOMES

- Plot landing outcomes at each launch site
- Green = success; red = failure
- Use MarkerCluster to identify outcome patterns

MarkerCluster • Green / Red

3

MEASURE PROXIMITY

- Use KSC LC-39A as an example launch site
- Draw colored lines to nearby rail, road, coast, and city
- Show distances and spatial relationships

Lines • Distances • Nearby features

FLOW

PLACE MARKERS → ADD CIRCLES → COLOR OUTCOMES → CLUSTER POINTS → DRAW DISTANCES

GitHub URL file: lab_jupyter_launch_site_location.ipynb

BUILD A DASHBOARD WITH PLOTLY DASH

Interactive exploration of Falcon 9 launch success by location, payload mass, and booster version

1

LAUNCH SITE FILTER

- Select All Sites or a specific launch site from a dropdown
- Focus success analysis on the selected location
- Update charts dynamically after selection

Dropdown • All Sites / Selected Site

2

SUCCESS PIE CHART

- All Sites: show successful launches by launch site
- Selected Site: compare success and failure counts
- Reveal overall performance at a glance

Plotly Pie • Success / Failure

3

PAYLOAD EXPLORATION

- Set payload-mass limits with a range slider
- Plot payload mass vs. launch outcome
- Color points by booster version to compare patterns

RangeSlider • Scatter Plot • Booster

FLOW

SELECT SITE → SET PAYLOAD → FILTER DATA → UPDATE PLOTS → COMPARE RESULTS

GitHub URL file: spacex-dash-app.py

PREDICTIVE ANALYSIS (CLASSIFICATION)

Predicting Falcon 9 booster landing success • Binary classification: 1 = success, 0 = failure

01 PREPARE FEATURES & LABELS

TARGET LABELS

Extract Class → NumPy y

Binary landing outcome

→

TRAIN / TEST SPLIT

train_test_split(X, y)

Hold out unseen test records

→

STANDARDIZE

StandardScaler

Fit on training data only

02 OPTIMIZE FOUR CLASSIFIERS

GRIDSEARCHCV

10-fold cross-validation
(cv = 10)

→

LOGISTIC REGRESSION

C, penalty

SUPPORT VECTOR MACHINE

C, kernel

DECISION TREE

max_depth

K-NEAREST NEIGHBORS

n_neighbors

03 EVALUATE & SELECT THE BEST MODEL

TEST-SET ACCURACY

Compare method.score() across models

→

CONFUSION MATRICES

Review errors, precision, recall & F1

→

SELECT BEST MODEL

Rank test accuracy; check overfitting

GitHub URL file: SpaceX_Machine Learning Prediction_Part_5.ipynb

RESULTS

Exploring launch performance through data, visualization, and predictive models

1

EXPLORATORY DATA ANALYSIS RESULTS

Launch patterns and trends

Relationships across payload, orbit, site, and outcomes

EDA • SQL • DATA VISUALIZATION

2

INTERACTIVE VISUAL ANALYTICS & DASHBOARD

Geographic launch-site insights

Interactive filters, maps, landing-success views, and screenshots

FOLIUM • PLOTLY DASH

3

PREDICTIVE ANALYSIS RESULTS

Classification model comparison

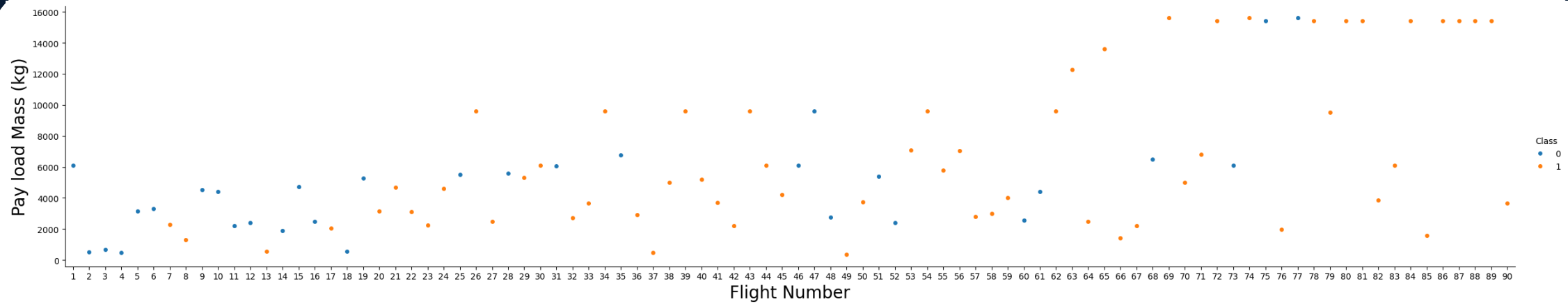
Evaluation of landing-outcome predictions

LR • SVM • TREE • KNN

Detailed findings and charts are presented in the following slides

FLIGHT NUMBER VS. PAYLOAD MASS

How launch experience and payload relate to landing outcome



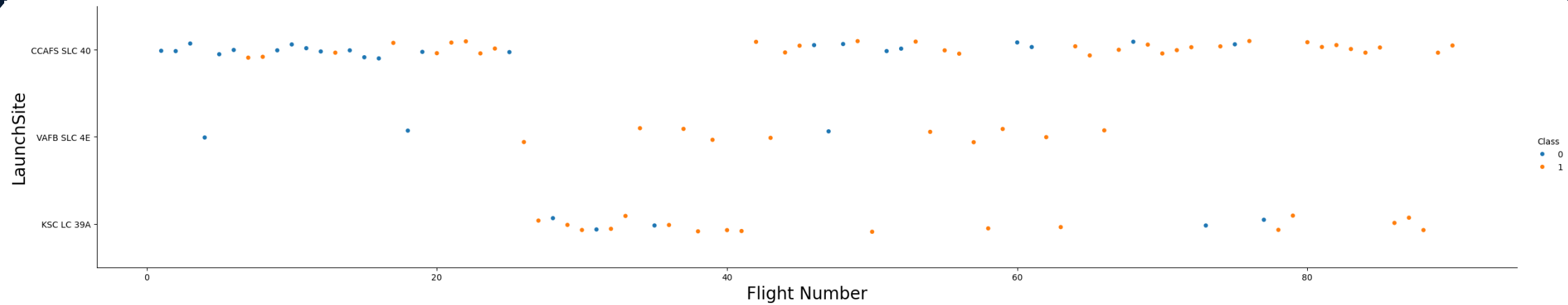
KEY OBSERVATIONS

- Landing successes are more common in later flight numbers than in the earliest flights.
- Payload mass varies widely across flights, and both outcomes occur across much of the range.
- Flight experience appears relevant; this chart alone does not establish a payload threshold.

Results based on running edadataviz.ipynb | Class 1 = successful booster landing; Class 0 = unsuccessful

FLIGHT NUMBER VS. LAUNCH SITE

When and where successful booster landings emerge



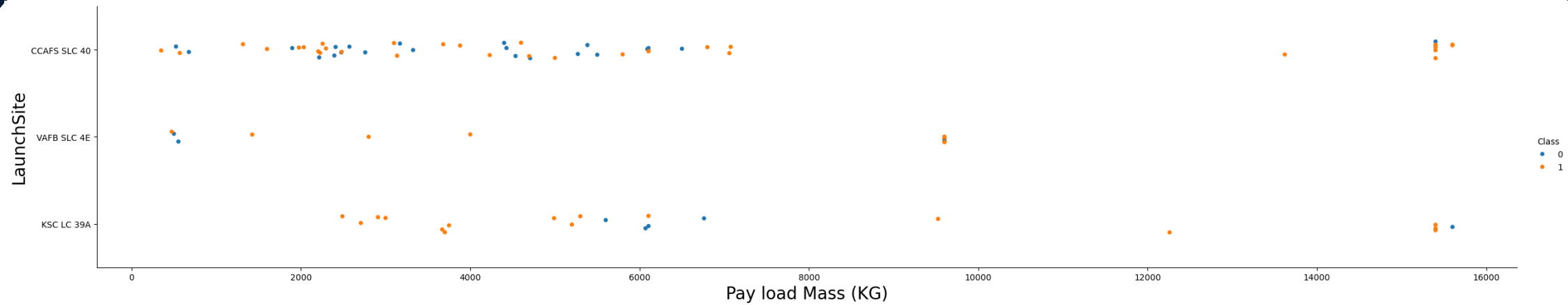
KEY OBSERVATIONS

- CCAFS SLC-40 accounts for many of the early flights, with more failures in the early sequence.
- KSC LC-39A appears from later flight numbers and shows predominantly successful outcomes.
- VAFB SLC-4E has fewer observations; success patterns also depend on launch chronology.

Results based on edadataviz.ipynb | Class 1 = successful booster landing; Class 0 = unsuccessful

PAYLOAD MASS VS. LAUNCH SITE

How payload distribution differs across launch facilities



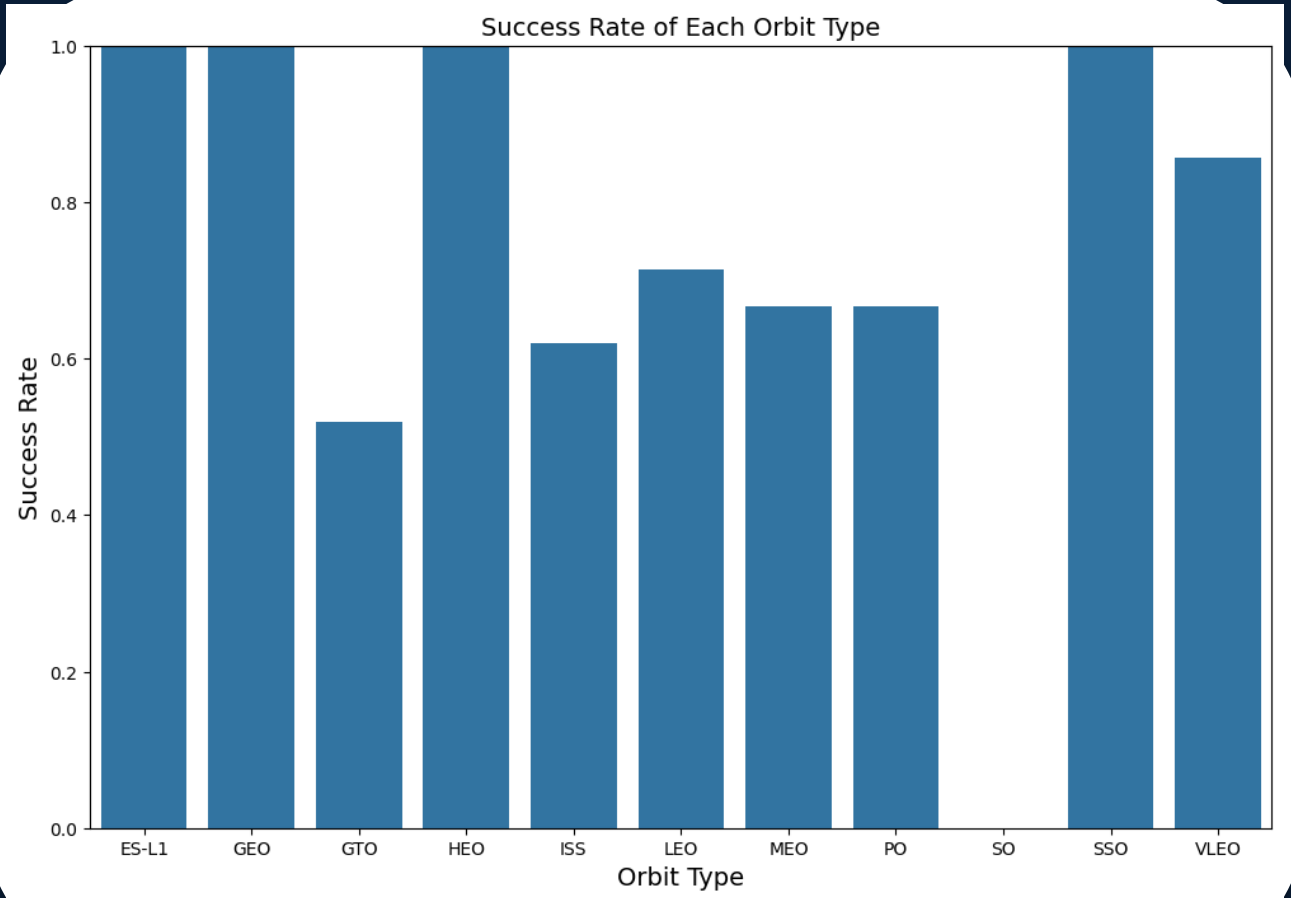
KEY OBSERVATIONS

- KSC LC-39A and CCAFS SLC-40 cover a broad payload range, including very heavy payloads.
- VAFB SLC-4E has no payload above 10,000 kg in the plotted sample.
- Successful and failed landings overlap in payload mass at several sites.

Results based on running edadataviz.ipynb | Class 1 = successful booster landing; Class 0 = unsuccessful

ORBIT TYPE VS. SUCCESS RATE

Average Class = 1 landing rate by orbit category



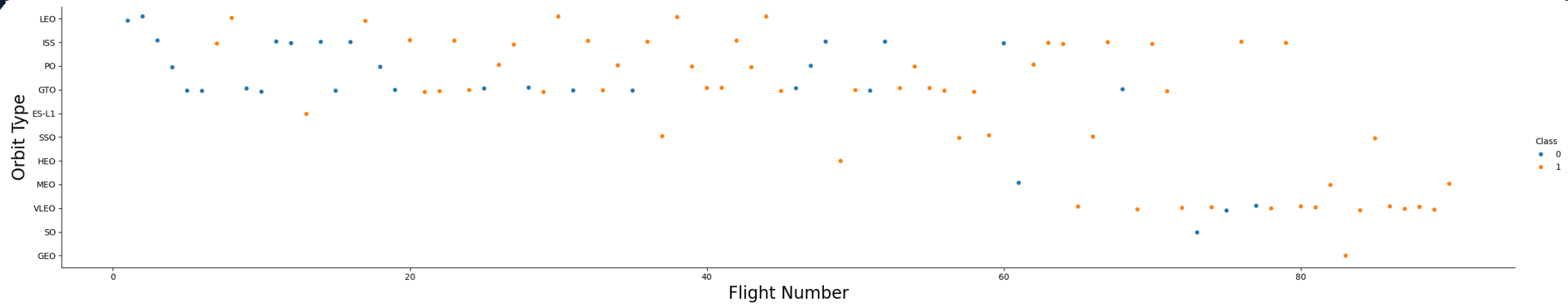
KEY OBSERVATIONS

- ES-L1, GEO, LEO and SSO reach 100% success in the plotted observations.
- VLEO is also high (about 85%), while GTO and several other orbits are lower.
- Orbit-level rates may reflect unequal sample sizes and flight histories; interpret extremes cautiously.

Results based on running edadataviz.ipynb | Class 1 = successful booster landing; Class 0 = unsuccessful

FLIGHT NUMBER VS. ORBIT TYPE

Landing outcomes across flight sequences and orbit types



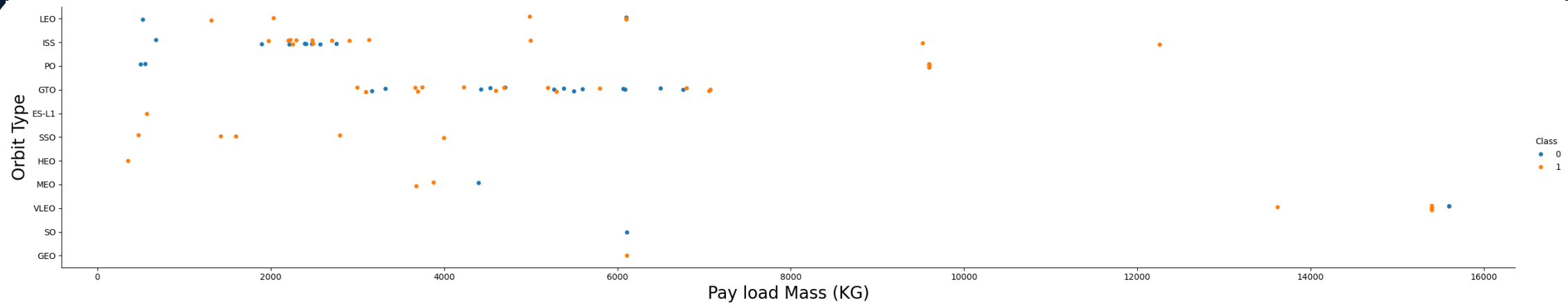
KEY OBSERVATIONS

- Later flights introduce many VLEO observations, most with successful landing outcomes.
- For LEO, successful outcomes become more apparent at later flight numbers.
- GTO contains both successful and unsuccessful landings over multiple flight numbers.

Results based on running edadataviz.ipynb | Class 1 = successful booster landing; Class 0 = unsuccessful

PAYLOAD MASS VS. ORBIT TYPE

Landing outcomes across payload ranges and mission orbits



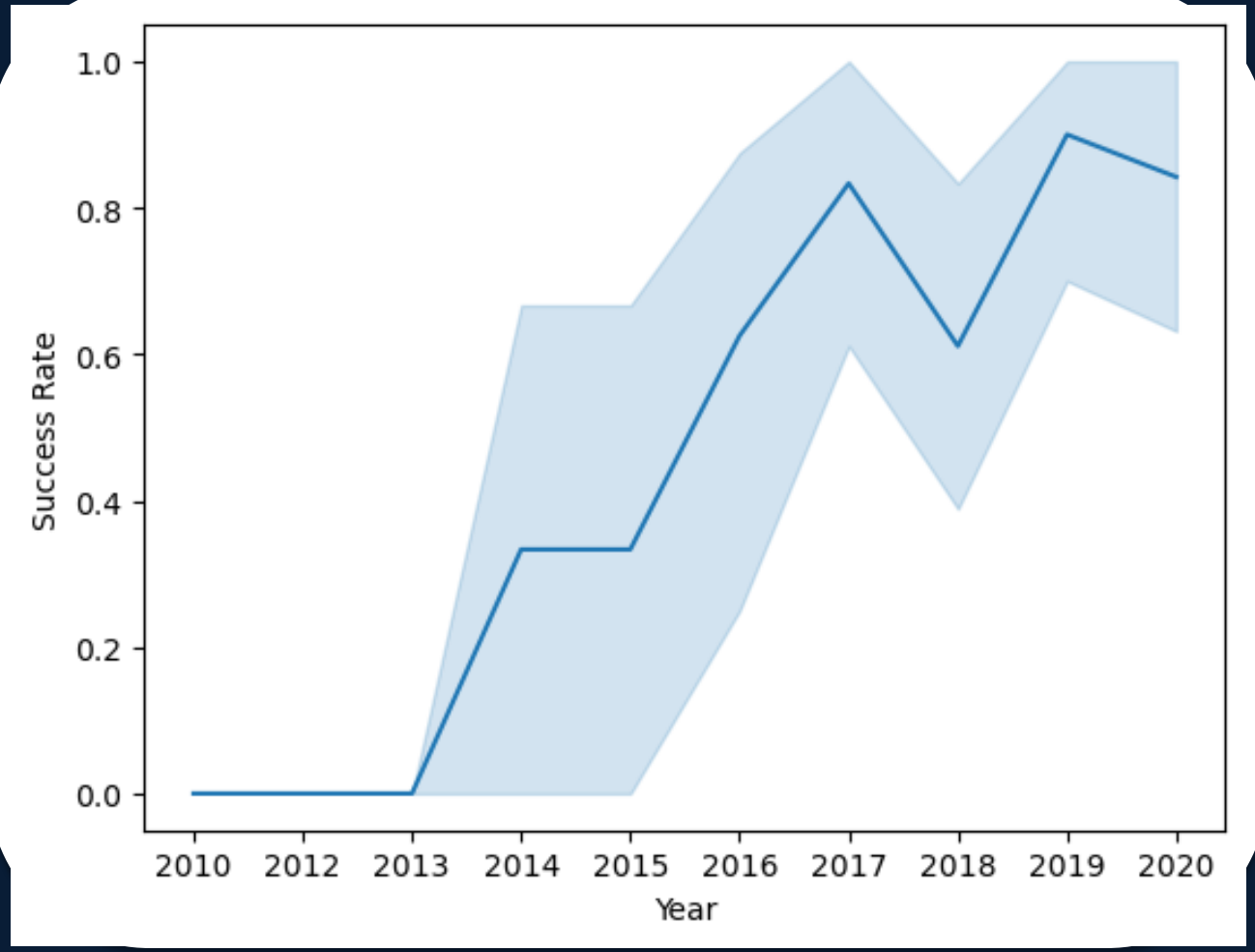
KEY OBSERVATIONS

- GTO spans a substantial payload range and includes both outcomes, limiting simple separation.
- LEO and ISS show several successful outcomes, including at relatively high payload masses.
- Orbit and payload may jointly influence prediction, but the scatter plot is descriptive.

Results based on running edadataviz.ipynb | Class 1 = successful booster landing; Class 0 = unsuccessful

YEAR VS. LANDING SUCCESS RATE

Yearly average of the binary landing-success label



KEY OBSERVATIONS

- Annual success rate is near zero in the early years and rises substantially from about 2014.
- The plotted rate reaches roughly 80–90% in several later years, with fluctuations.
- The shaded interval suggests uncertainty; annual sample counts should be checked before comparing years.

Results based on running edadataviz.ipynb | Class 1 = successful booster landing; Class 0 = unsuccessful

EDA WITH SQL | UNIQUE LAUNCH SITES

TASK 01 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT DISTINCT "Launch_Site" from SPACEXTBL;
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

WHAT THE RESULT SHOWS

Four distinct launch-site identifiers appear in the SQL dataset. They provide the geographic categories for subsequent site-level analysis.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | LAUNCH SITES BEGINNING WITH CCA

TASK 02 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT "Launch_site" from SPACEXTBL where "Launch_site" LIKE 'CCA%'
LIMIT 5;
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

Launch_Site
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40
CCAFS LC-40

WHAT THE RESULT SHOWS

The first five matches all belong to CCAFS LC-40. LIMIT 5 returns rows, not distinct sites, so repeated values are expected.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | NASA (CRS) TOTAL PAYLOAD MASS

TASK 03 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT sum(PAYLOAD_MASS__KG_) AS totpayloadmass from SPACEXTBL
WHERE Customer='NASA (CRS)';
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

totpayloadmass
45596

WHAT THE RESULT SHOWS

NASA Commercial Resupply Services (CRS) missions account for 45,596 kg of payload in this dataset; this is their aggregate mass, not the mass of one launch.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | F9 V1.1 AVERAGE PAYLOAD MASS

TASK 04 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT avg(PAYLOAD_MASS__KG_) as avgpayloadmass from SPACEXTBL
WHERE Booster_Version='F9 v1.1';
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

avgpayloadmass
2928.4

WHAT THE RESULT SHOWS

The mean payload for booster version F9 v1.1 is 2,928.4 kg across the matching records, offering a baseline for comparing rocket generations.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | FIRST SUCCESSFUL GROUND-PAD LANDING

TASK 05 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT min(DATE) from SPACEXTBL
WHERE Landing_outcome='Success (ground pad)';
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

min(DATE)
2015-12-22

WHAT THE RESULT SHOWS

The earliest recorded successful landing on a ground pad is 22 December 2015. This is a milestone in first-stage recovery.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | SUCCESSFUL DRONE-SHIP LANDINGS: 4–6 T

TASK 06 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT "Booster_Version" from SPACEXTBL
WHERE Landing_Outcome='Success (drone ship)' and PAYLOAD_MASS__KG_>4000 and  PAYLOAD_MASS__KG_< 6000;
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

WHAT THE RESULT SHOWS

Four booster-version records satisfy both conditions: a successful drone-ship landing and a payload strictly between 4,000 and 6,000 kg.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | MISSION OUTCOMES: SUCCESS VS FAILURE

TASK 07 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT SUM(CASE WHEN Mission_Outcome LIKE '%Success%'THEN 1 ELSE 0 END) as total_success_mission, SUM(CASE WHEN
Mission_Outcome LIKE '%Failure%'THEN 1 ELSE 0 END)
  as total_failure_mission
FROM SPACEXTBL;
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

total_success_mission	total_failure_mission
100	1

WHAT THE RESULT SHOWS

The mission-outcome field records 100 successes and 1 failure. These are mission outcomes—not booster landing outcomes—and must not be confused with landing success.

NOTE: Mission success and booster-landing success measure different events.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | BOOSTER VERSIONS WITH MAXIMUM PAYLOAD

TASK 08 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT DISTINCT "Booster_version"
FROM SPACEXTBL
WHERE PAYLOAD_MASS__KG_= (SELECT MAX(PAYLOAD_MASS__KG_)
FROM SPACEXTBL);
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

WHAT THE RESULT SHOWS

Twelve distinct booster-version labels tie at the maximum payload mass. The query returns all tied booster versions rather than arbitrarily choosing one.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | DRONE-SHIP LANDING FAILURES IN 2015

TASK 09 SQL command → notebook output → interpretation

SQL QUERY

```
SELECT CASE substr(Date,6,2)
 WHEN '01' THEN 'January'
 WHEN '02' THEN 'February'
 WHEN '03' THEN 'March'
 WHEN '04' THEN 'April'
 WHEN '05' THEN 'May'
 WHEN '06' THEN 'June'
 WHEN '07' THEN 'July'
 WHEN '08' THEN 'August'
 WHEN '09' THEN 'September'
 WHEN '10' THEN 'October'
 WHEN '11' THEN 'November'
 WHEN '12' THEN 'December'
 END AS Month_Name, "Landing_Outcome", "Booster_Version", "Launch_Site"
 FROM SPACEXTBL
 WHERE substr(Date, 1, 4) = '2015' AND "Landing_Outcome" = 'Failure (drone ship)';
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

Month_Name	Landing_Outcome	Booster_Version	Launch_Site
January	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
April	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

WHAT THE RESULT SHOWS

The query identifies two failed drone-ship landings in 2015, both launched from CCAFS LC-40: one in January and one in April.

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

EDA WITH SQL | LANDING OUTCOMES RANKED BY FREQUENCY

TASK 10 SQL command → notebook output → interpretation

SQL QUERY

```
select "Landing_outcome", count ("Landing Outcome") as total from SPACEXTBL where date between "2010-06-04" and "2017-03-20" group by "Landing_Outcome" order by total desc;
```

SQL OUTPUT · SAVED NOTEBOOK RESULT

Landing_Outcome	total
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

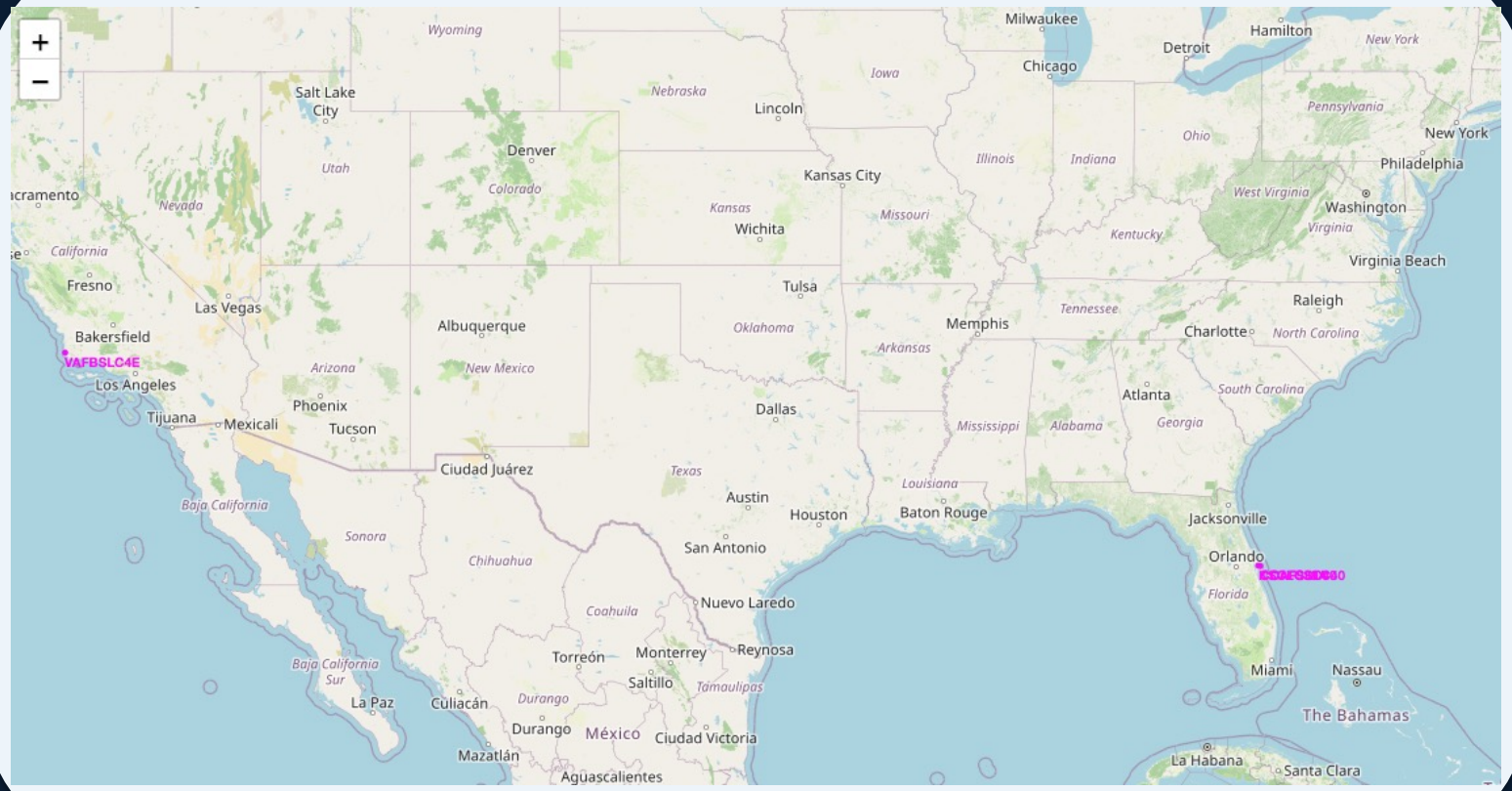
WHAT THE RESULT SHOWS

“No attempt” is the most common category in the saved result (10).

Results based on running jupyter-labs-eda-sql-coursera_sqllite.ipynb

FOLIUM RESULTS — LAUNCH SITE LOCATIONS

Four launch sites and NASA Johnson Space Center as a reference



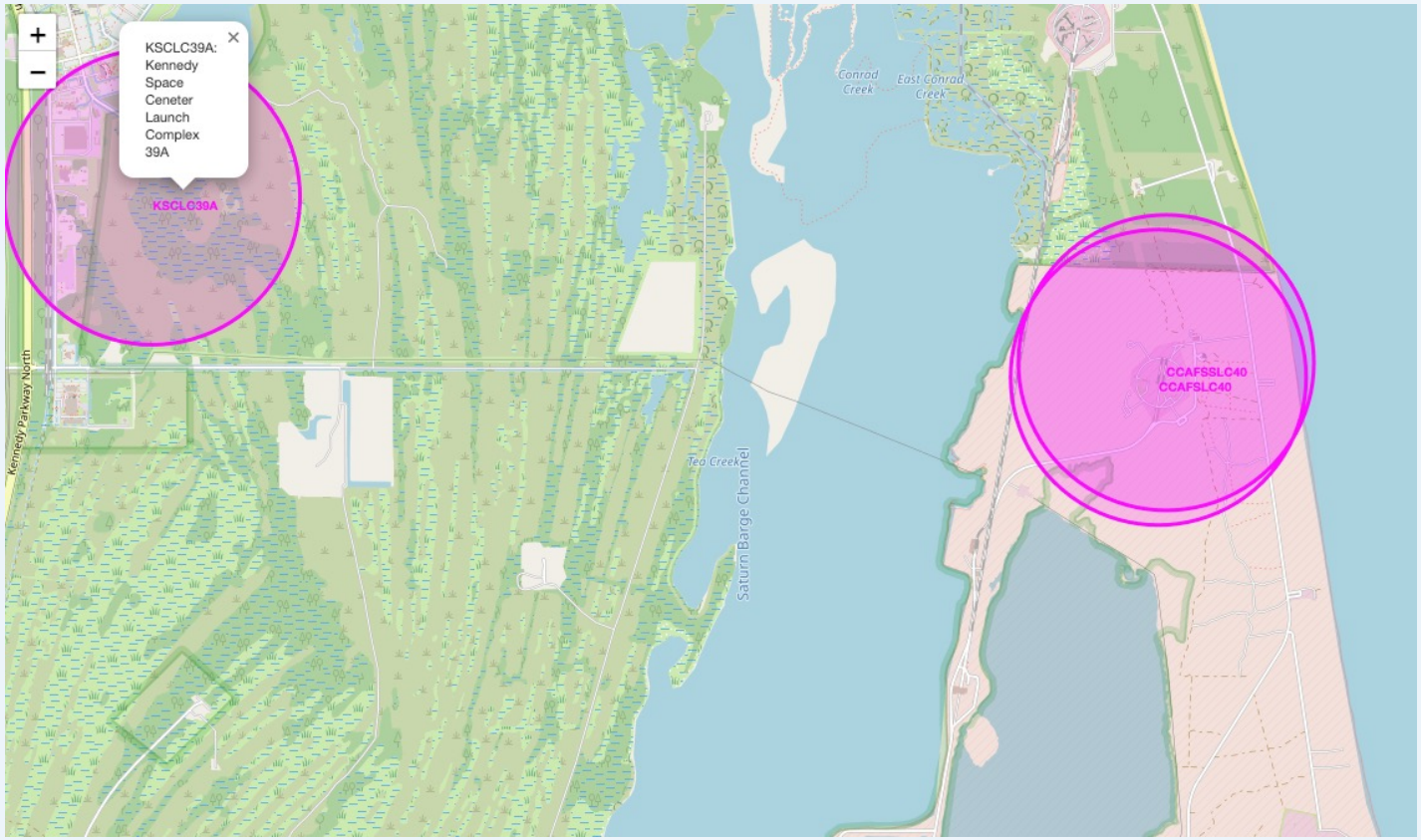
Geographic insights

- The four launch identifiers lie on U.S. Atlantic or Pacific coastal regions.
- Three sites cluster near Cape Canaveral, Florida; one is on California's coast.
- No site is close to the equator; coastal position is the clearer common pattern.

Results based on running lab_jupyter_launch_site_location.ipynb

FOLIUM RESULTS — SITE MARKERS & LABELS

Circle and popup markers distinguish launch-site coordinates



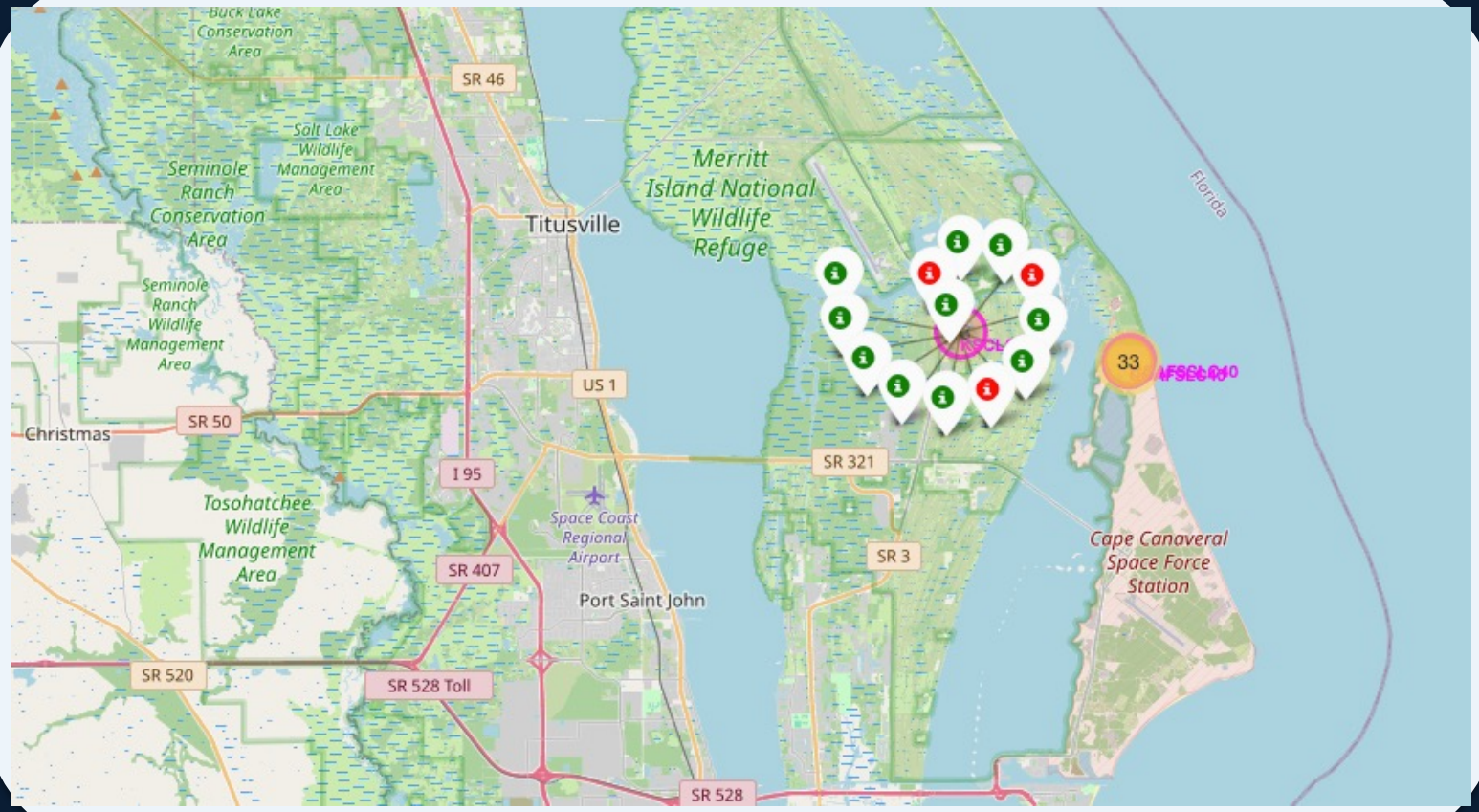
Map interpretation

- Florida’s three identifiers occupy a compact geographic region.
- CCAFS LC-40 and CCAFS SLC-40 appear almost coincident at this map scale.
- Zoom and popup labels are needed to distinguish nearby launch locations.

Results based on running lab_jupyter_launch_site_location.ipynb

FOLIUM RESULTS — LANDING OUTCOME MARKERS

Green = successful booster landing; red = unsuccessful



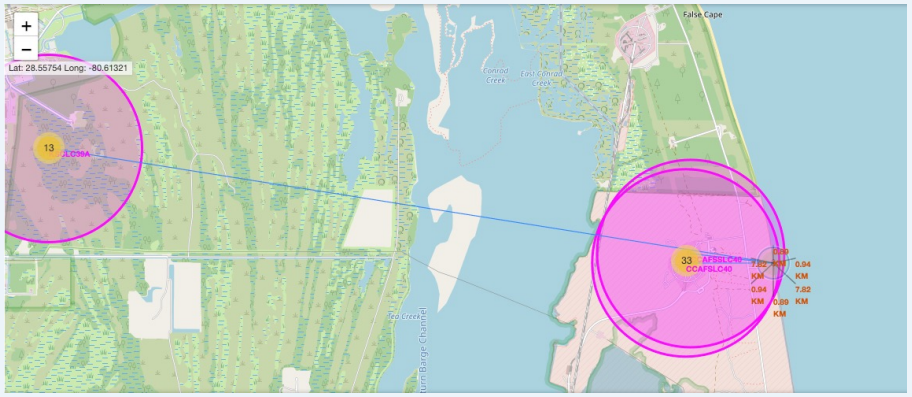
Observed outcomes

- KSC LC-39A: 10 of 13 landings successful (76.9%), highest among the four groups.
- CCAFS LC-40: 7 of 26 (26.9%); other sites fall between those rates.
- MarkerCluster groups overlapping launch records; comparison needs site-level counts.

Results based on running lab_jupyter_launch_site_location.ipynb

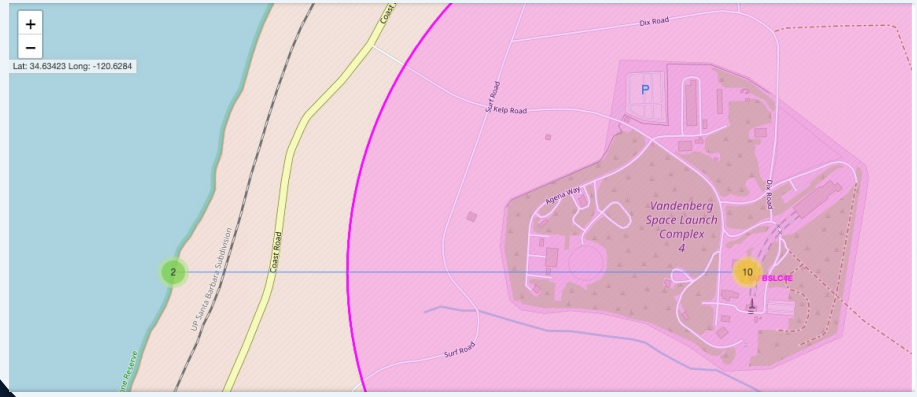
FOLIUM RESULTS — COASTLINE DISTANCES

Haversine distances between site coordinates and notebook-selected coastal points



FLORIDA

KSC LC-39A and Cape Canaveral selected coastline distances



VANDENBERG

SLC-4E and selected Pacific coastline point

Distance findings

- Selected-point distances: CCAFS LC-40 0.94 km; CCAFS SLC-40 0.89 km.
- VAFB SLC-4E is 1.43 km from its chosen point; KSC LC-39A is 7.82 km.
- These are not independently verified shortest distances to the actual coast.

Results based on running lab_jupyter_launch_site_location.ipynb

PLOTLY DASHBOARD RESULTS

Interactive visual analytics of SpaceX booster landing outcomes

01 Site selection

Four launch sites and All Sites

02 Success pie charts

Distribution across sites or within one site

03 Payload exploration

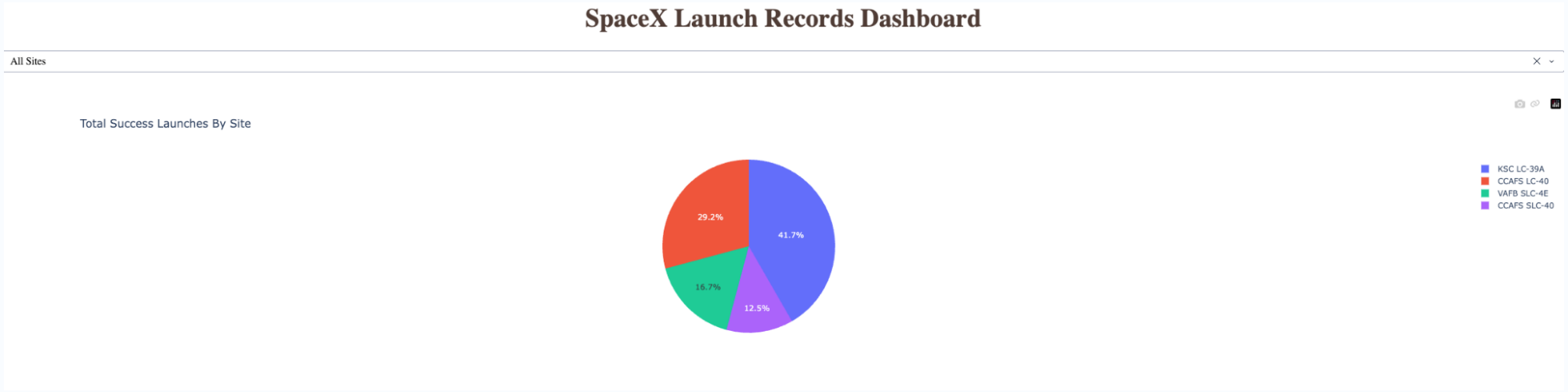
Slider-filtered scatter plots by booster version

Seven original dashboard screenshots • 56 launches • 24 successful booster landings

PLOTLY DASH SCREENSHOTS OBTAINED BY EXECUTING THE FILE spacex-dash-app.py in a terminal using python3.8 and launching an application port to view results in a browser

PLOTLY DASH RESULTS — ALL LAUNCH SITES

Success pie chart generated for the All Sites dropdown selection



KSC leads the share

KSC LC-39A contributes 41.7% of all successful booster landings: 10 of the 24 successes.

Successes by site

CCAFS LC-40: 7 (29.2%); VAFB SLC-4E: 4 (16.7%); CCAFS SLC-40: 3 (12.5%).

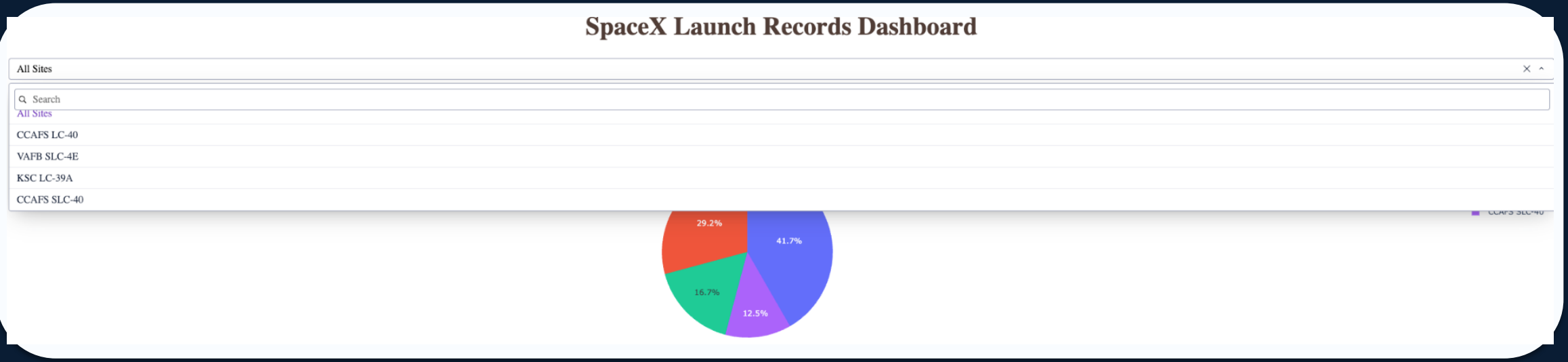
Read percentages carefully

These slices compare the distribution of successful landings, not each site's success probability.

Results based on executing spacex-dash-app.py in a terminal

PLOTLY DASH RESULTS — SITE DROPDOWN

Selecting a launch site changes the pie-chart analysis via a Dash callback



Four site options

The dropdown offers CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, and CCAFS SLC-40, plus All Sites.

Conditional display

All Sites shows each site’s contribution to total successes; a single-site choice shows success versus failure.

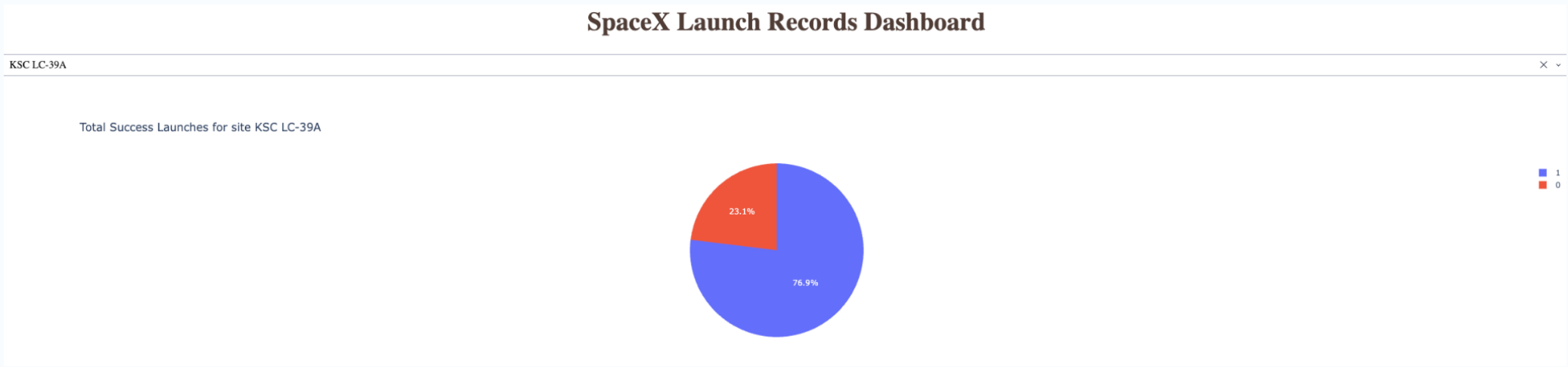
Analytical benefit

The control supports site-specific comparisons without manually recreating a separate chart for each site.

Results based on executing spacex-dash-app.py in a terminal

PLOTLY DASH RESULTS — KSC LC-39A

Selected-site pie chart: successful versus unsuccessful booster landings



76.9% success rate

Ten of 13 launches at KSC LC-39A resulted in a successful first-stage landing.

Three failed landings

The remaining 3 of 13 outcomes are failures (23.1%) in this dataset.

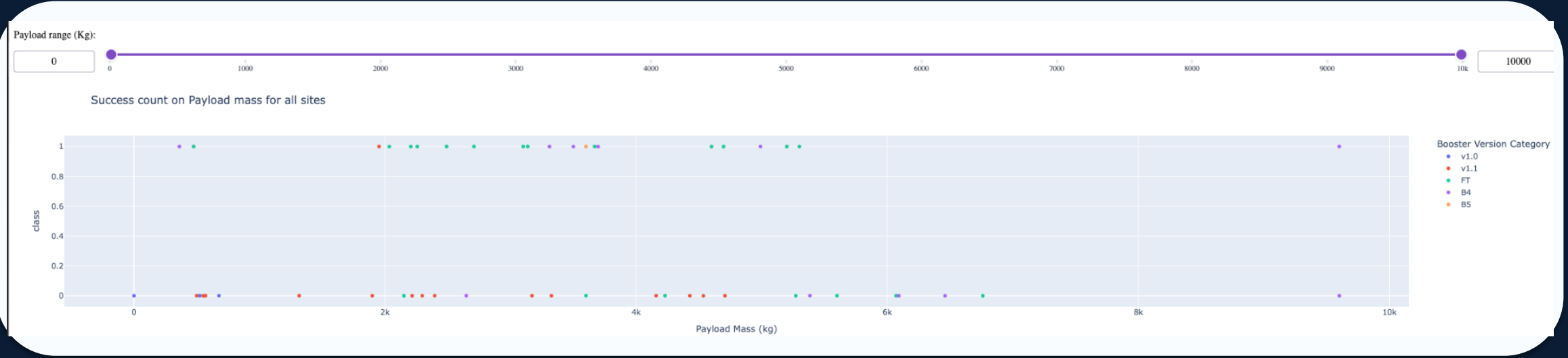
Compare the two pies

76.9% is KSC's own success rate; 41.7% on the All Sites chart is its share of all 24 successes.

Results based on executing spacex-dash-app.py in a terminal

PLOTLY DASH RESULTS — FULL PAYLOAD RANGE

Scatter plot: payload mass versus binary landing outcome, colored by booster version



Two outcome classes

Class = 1 indicates a successful booster landing; Class = 0 indicates an unsuccessful landing.

Multiple booster versions

Versions v1.0, v1.1, FT, B4 and B5 appear in the visualization, revealing changes across hardware generations.

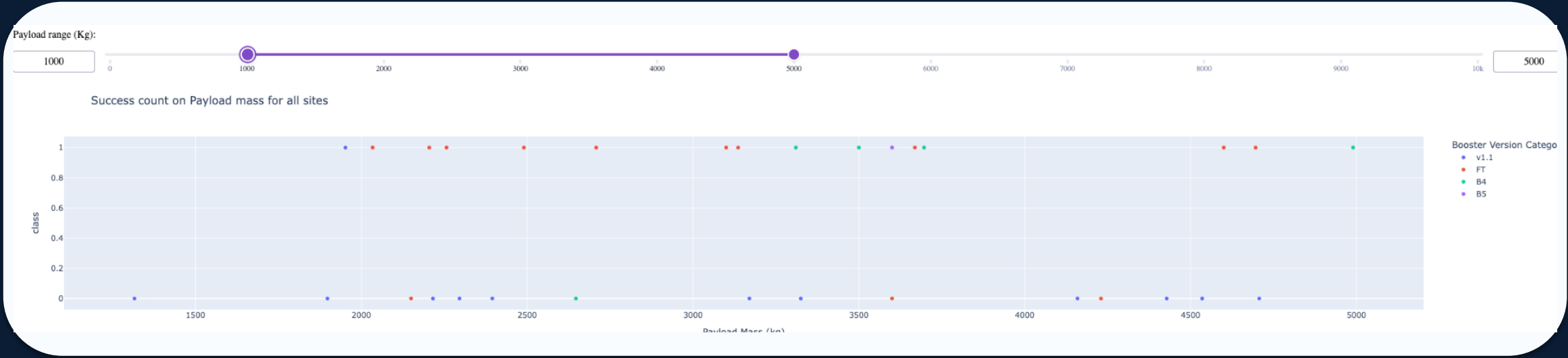
No simple mass cutoff

Successes and failures occur across overlapping payload masses; payload alone does not determine the outcome.

Results based on executing spacex-dash-app.py in a terminal

PLOTLY DASH RESULTS — 1,000–5,000 KG

Payload slider focuses the scatter plot on lower-to-moderate payload masses



Focused exploration

Restricting the slider reduces the displayed records to launches carrying 1,000–5,000 kg.

Mixed outcomes

Both landing successes and failures appear in this interval, even at similar payload masses.

Booster categories matter

The displayed versions differ across observations; assessing payload alongside booster version is more informative.

Results based on executing spacex-dash-app.py in a terminal

PLOTLY DASH RESULTS — 5,000–10,000 KG

Payload slider highlights the heavier payload subset



Smaller sample

The high-payload subset contains visibly fewer launches than the full-range view.

Predominantly failures

Most plotted heavy-payload observations in this screenshot are at Class = 0, with some successes.

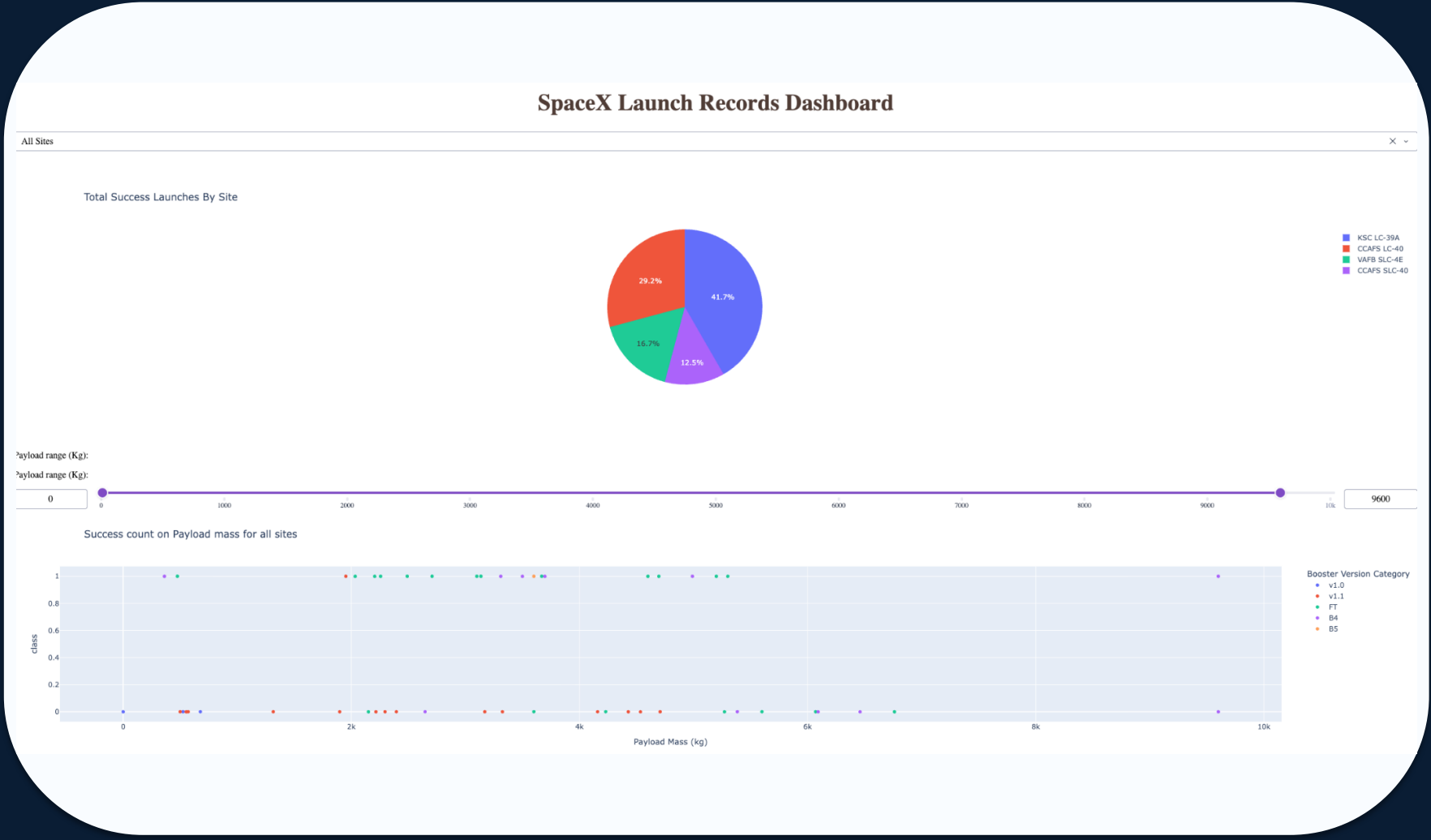
Interpret cautiously

This is exploratory association, not evidence that heavier payload directly causes landing failure.

Results based on executing spacex-dash-app.py in a terminal

PLOTLY DASH — INTERACTIVE RESULTS OVERVIEW

Complete dashboard with site selector, success pie chart, payload slider and outcome scatter



FOUR TASKS

01 Dropdown

Choose All Sites or one of four launch sites.

02 Pie callback

Refresh success distribution or site outcomes.

03 Range slider

Select minimum and maximum payload mass.

04 Scatter callback

Update payload/outcome points by booster version.

Results based on executing spacex-dash-app.py in a terminal

PREDICTIVE ANALYSIS — CLASSIFICATION RESULTS

Falcon 9 first-stage landing prediction | Four tuned classifiers

Training cases
72

Test cases
18

Predictor columns
83

Held-out test accuracy



Main finding

- Decision Tree: 17 of 18 correct
- Other models: 15 of 18 correct
- One test case changes accuracy
- by 5.6 percentage points

Results based on running SpaceX_Machine Learning Prediction_Part_5.ipynb

TRAINING PIPELINE & HYPERPARAMETER SEARCH

Notebook settings and a data-leakage caution

TARGET

Class = 1 landed

Class = 0 did not land

FEATURES

83 numeric / encoded

model input columns

SPLIT

80% train (72)

20% test (18)

TUNING

GridSearchCV

10-fold CV

IMPORTANT: STANDARDIZATION BEFORE SPLITTING

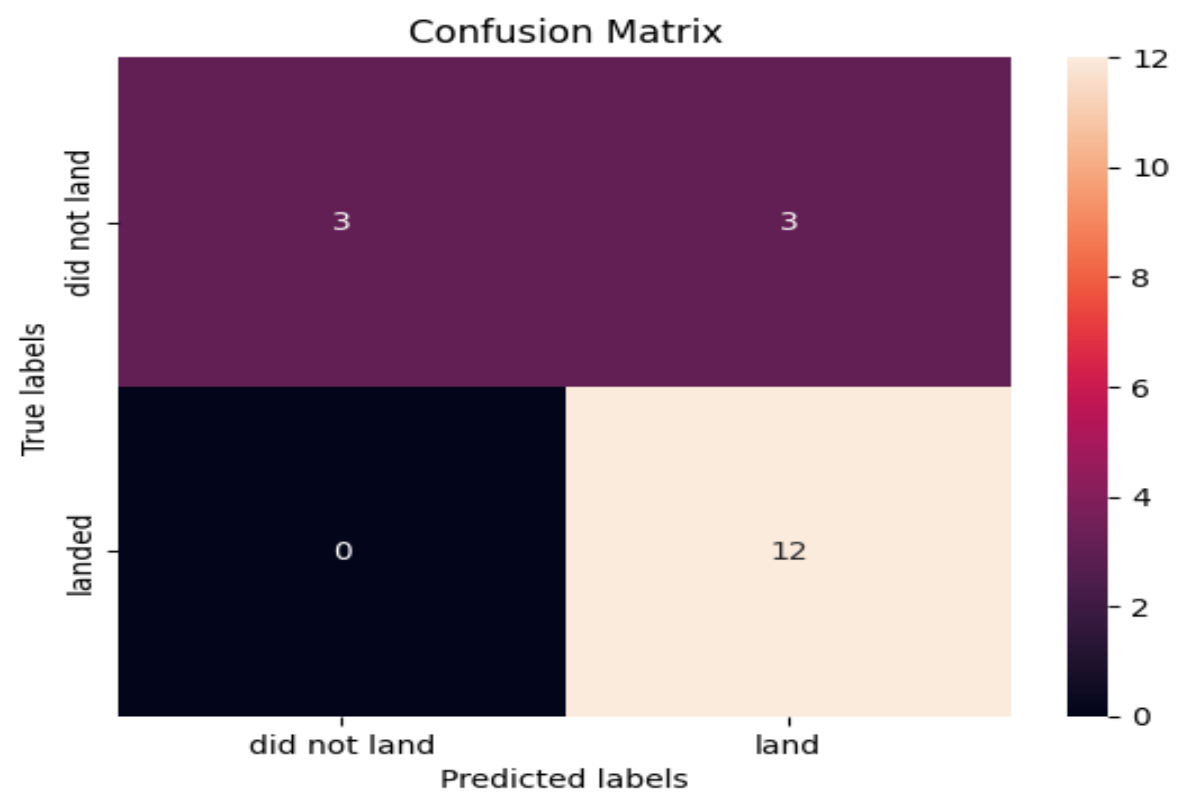
- The notebook fits StandardScaler on the full feature matrix before train_test_split.
- Better evaluation: split first; fit scaling within a Pipeline using training folds only.
- The reported accuracy values are preserved here; leakage-safe scores may differ.

Results based on running SpaceX_Machine Learning Prediction_Part_5.ipynb

LOGISTIC REGRESSION

Optimized model performance | Saved notebook confusion matrix

Test set confusion matrix



Test accuracy

83.3%

10-fold best CV score

84.6%

Best parameters & insight

C = 0.01 • penalty = l2 • solver = lbfgs

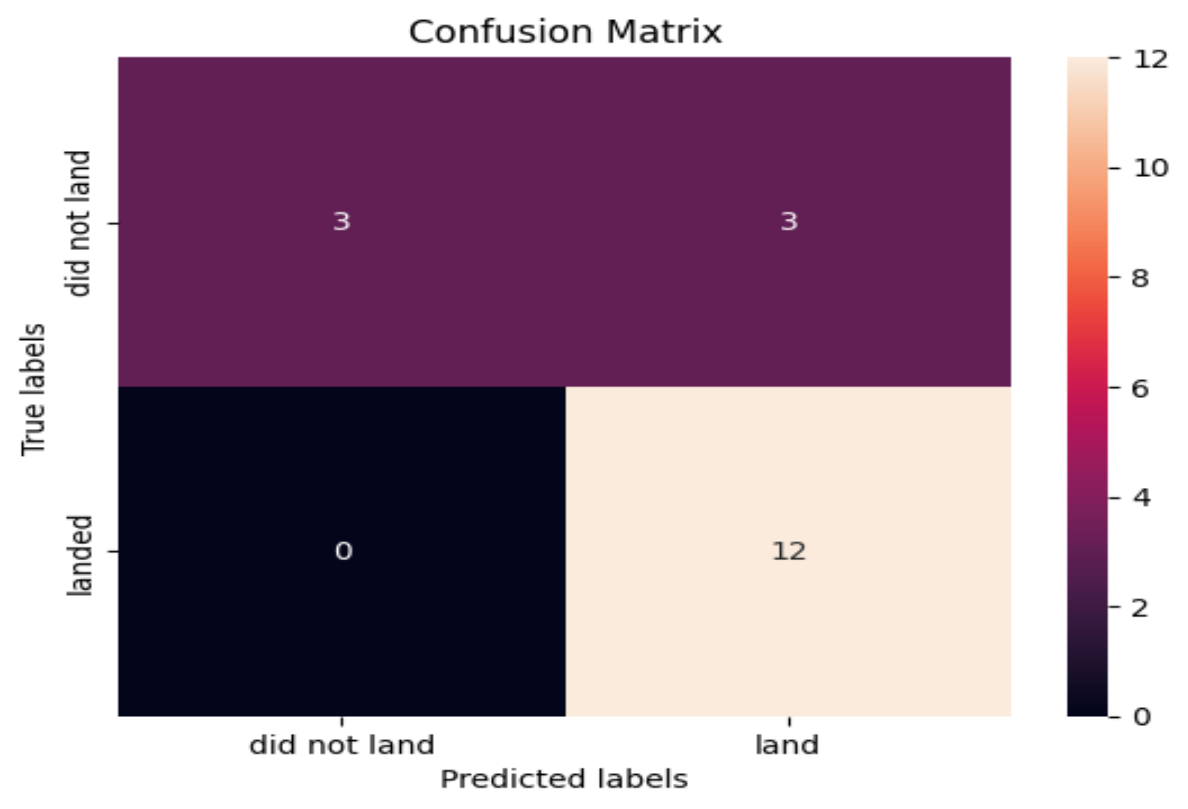
3 false positives; all 12 successful landings detected.

Results based on running SpaceX_Machine Learning Prediction_Part_5.ipynb

SUPPORT VECTOR MACHINE

Optimized model performance | Saved notebook confusion matrix

Test set confusion matrix



Test accuracy

83.3%

10-fold best CV score

84.8%

Best parameters & insight

C = 1 • gamma = 0.03162 • kernel = sigmoid

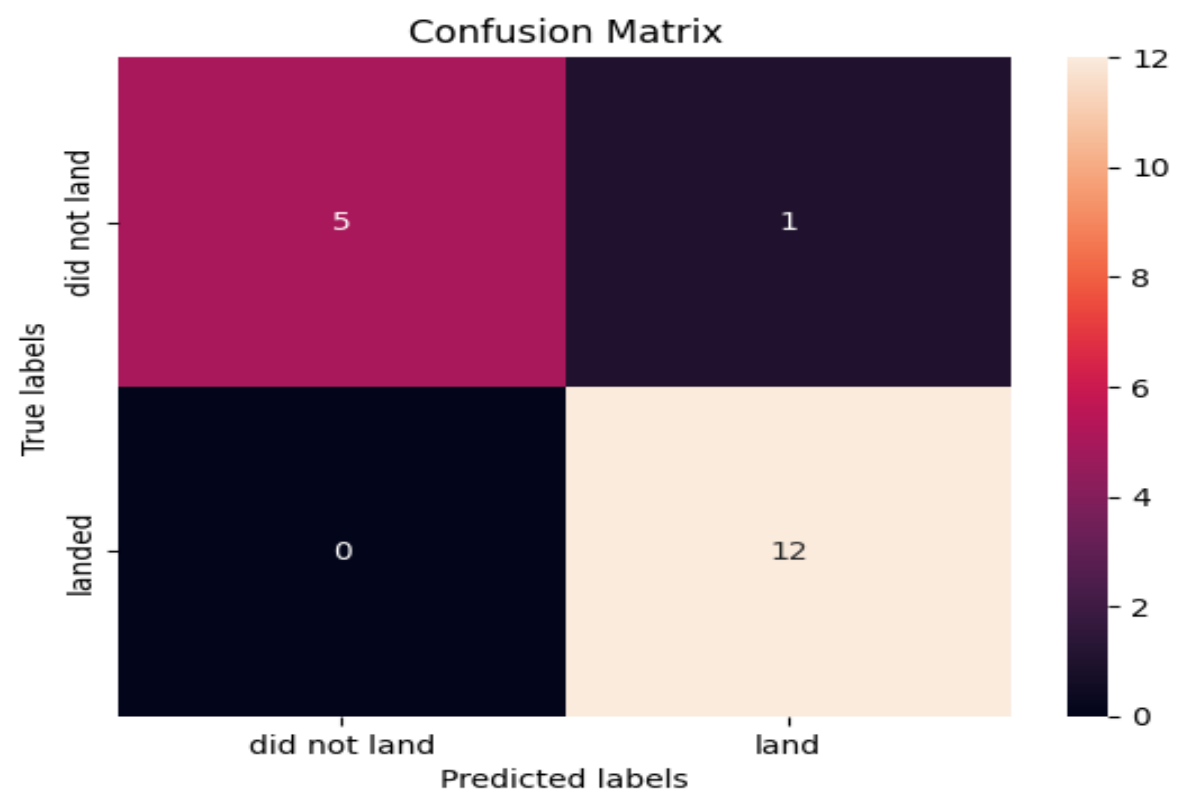
Same test confusion matrix as Logistic Regression.

Results based on running SpaceX_Machine Learning Prediction_Part_5.ipynb

DECISION TREE

Optimized model performance | Saved notebook confusion matrix

Test set confusion matrix



Test accuracy

94.4%

10-fold best CV score

86.3%

Best parameters & insight

gini • max_depth = 8 • max_features = sqrt

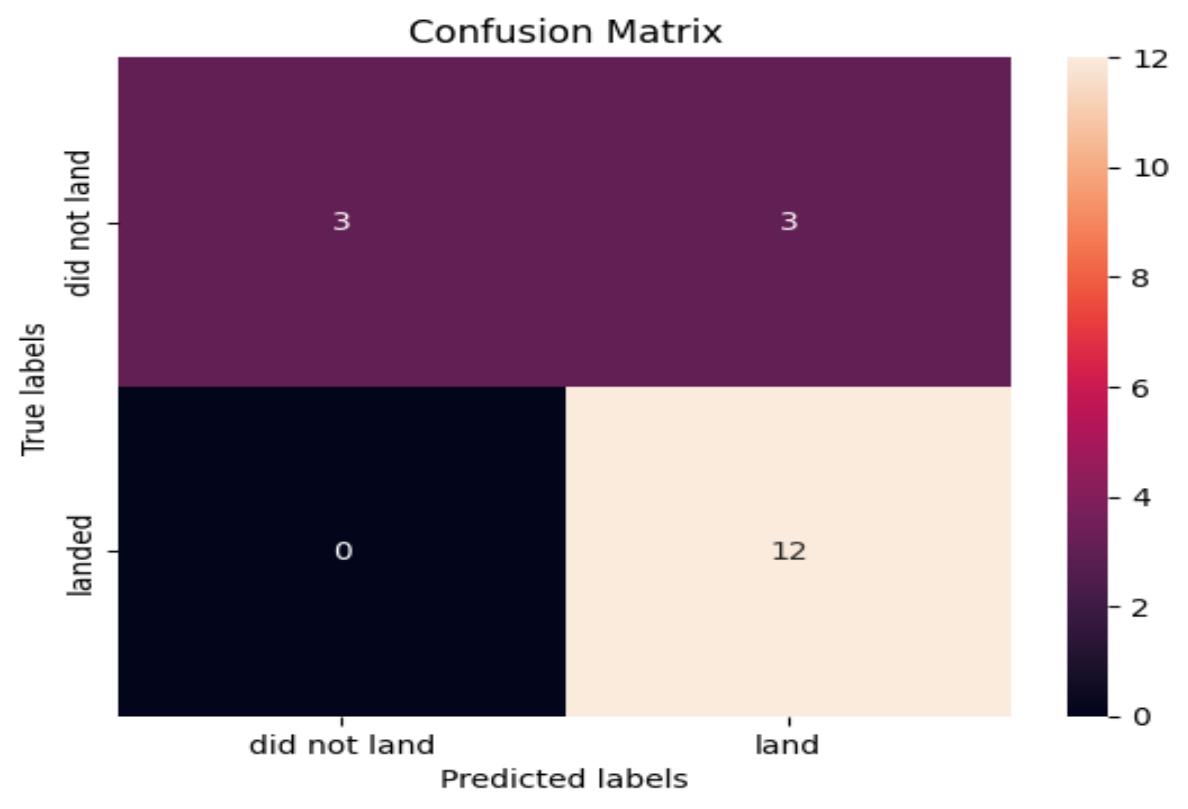
Five of six unsuccessful landings correctly flagged; one false positive.

Results based on running SpaceX_Machine Learning Prediction_Part_5.ipynb

K-NEAREST NEIGHBORS

Optimized model performance | Saved notebook confusion matrix

Test set confusion matrix



Test accuracy

83.3%

10-fold best CV score

84.8%

Best parameters & insight

n_neighbors = 10 • p = 1 • algorithm = auto

Three of six unsuccessful landings misclassified as successful.

Results based on running SpaceX_Machine Learning Prediction_Part_5.ipynb

MODEL COMPARISON — WHICH PERFORMS BEST?

Cross-validation accuracy and held-out accuracy are not identical

Model results (saved notebook)

Classifier	CV score	Test score	Correct
Logistic Regression	84.6%	83.3%	15/18
SVM	84.8%	83.3%	15/18
Decision Tree	**86.3%**	**94.4%**	**17/18**
KNN	84.8%	83.3%	15/18

Interpretation

- Decision Tree ranks first on
- this particular held-out split.
- The CV-score advantage is
- much smaller than the test
- accuracy gap.
- Do not claim universal
- superiority from 18 tests.

Results based on running SpaceX_Machine Learning Prediction_Part_5.ipynb

CONCLUSION — KEY FINDINGS

What the SpaceX case study reveals across EDA, interactive visualization, and prediction

EXPLORATORY ANALYSIS

- Booster landing success generally improves across later flights and years.
- Outcomes vary by payload, launch site, and orbit; these are useful candidate predictors.

EDA • SQL • TIME TRENDS

INTERACTIVE ANALYTICS

- Folium highlights coastal launch-site geography and spatial proximity.
- Plotly Dash enables site-level and payload-range comparisons of landing outcomes.

FOLIUM • PLOTLY DASH

PREDICTIVE MODELING

- Decision Tree achieved 94.4% test accuracy (17/18) in the notebook's saved run.
- Logistic Regression, SVM, and KNN each achieved 83.3% (15/18).

GRIDSEARCHCV • FOUR CLASSIFIERS

CONCLUSION — INTERPRETATION & NEXT STEPS

Promising results, but predictive reliability requires further validation

MAIN TAKEAWAY **Public SpaceX launch data supports meaningful exploration and preliminary landing-outcome prediction.**

IMPORTANT LIMITATIONS

- Only 18 test cases: one prediction changes accuracy by 5.6 percentage points.
- The notebook's pre-split scaling can leak test information.
- Small samples and one split limit confidence in model rankings.

RECOMMENDED NEXT STEPS

- Fit StandardScaler within a training-only Pipeline.
- Repeat stratified splits or nested cross-validation; report uncertainty.
- Compare precision, recall, F1, and confusion matrices on independent data.

THANK YOU

SPECIAL THANKS

To all instructors and creators of the ten Data Science courses
in the IBM Data Science Professional Certificate program

WITH SPECIAL APPRECIATION TO

COURSERA **IBM**

Thank you for making accessible, practical data science education possible.